

DIPLOMATURA EN DISEÑO DE PROCESOS DE NEGOCIOS · FCE-UBA



De CheTunes a SintonIA

Rediseño de un proceso de descubrimiento musical con IA agéntica

Del disparo único a la conversación con memoria

Trabajo Final Integrador · Cohorte 2026

Alumno: Kevin Rico

Docente: Prof. Diego Parrás

Julio 2026

a. Introducción

Elegí este proceso porque ya lo había construido y sabía sus oportunidades de mejora. En diciembre de 2025 desarrollé **CheTunes**, una aplicación web de descubrimiento musical con IA generativa (Google Gemini). Funcionaba, estaba desplegada en la nube y cumplía sus objetivos, pero al usarla en el día a día detecté una fricción de fondo que ninguna mejora estética podía resolver: **la aplicación no recordaba nada**. El usuario cargaba todo su contexto de golpe, recibía una respuesta cerrada, y si quería ajustar algo tenía que empezar de cero.

Este Trabajo Final toma ese proceso ya digitalizado y lo **rediseña por completo**: migra la experiencia desde una web rígida basada en código (Gemini + Google AI Studio) hacia un **flujo conversacional continuo con memoria de contexto**, construido sobre **Dify** en formato Chatflow. El resultado es **SintonIA**, un chatbot musical que mantiene el hilo de la conversación, hace preguntas de seguimiento y enruta automáticamente cada mensaje a la función correcta. Cabe destacar que la página de CheTunes sigue activa, con la diferencia de que ahora se le incluye el chatbot.

Regla de oro aplicada: no automaticé un proceso malo. CheTunes ya era un proceso validado y funcional; lo que hice fue diagnosticar su fricción real (la rigidez del disparo único) y rediseñar específicamente esa capa, no reconstruir de cero.

Objetivos del rediseño

- **Principal:** transformar una experiencia web de un solo disparo en un flujo conversacional continuo con memoria de sesión, sin perder ninguna de las tres funcionalidades originales.
- **Específico 1:** incorporar memoria de contexto para que el usuario construya su pedido de a poco, en diálogo, en lugar de cargar todo de una vez.
- **Específico 2:** reemplazar la navegación manual entre pestañas por un clasificador de intenciones que derive cada mensaje a la función adecuada.
- **Específico 3:** reducir la complejidad técnica de mantenimiento, pasando de código (TypeScript + API) a una plataforma visual de bajo código.

b. El proceso, antes (CheTunes)

CheTunes resolvía tres tareas del descubrimiento musical: recomendar canciones, identificar portadas de álbumes por foto y analizar letras. Técnicamente era sólido —usaba doble modelo Gemini, visión por computadora y búsqueda en tiempo real para evitar alucinaciones— pero su **arquitectura de interacción** era la de una web tradicional de formularios.

Disparador y flujo original

DISPARADOR: el usuario entra a la web y elige manualmente una de 3 pestañas

- Pestaña "Recomendar" → completa un formulario de una sola vez (artistas + actividad) → recibe 3 canciones → FIN
- Pestaña "Escanear" → sube una foto → recibe ficha del álbum → FIN
- Pestaña "Analizar" → pega una letra → recibe análisis → FIN

RESULTADO: respuesta cerrada. Para ajustar algo, se reinicia el formulario.

Responsables del proceso (antes)

Actor	Rol en el proceso
Usuario	Elige la pestaña correcta, carga todo el contexto de una vez, interpreta la respuesta cerrada.
Modelo Gemini	Procesa el formulario en una única llamada y devuelve un resultado sin continuidad.
Código frontend (TS)	Orquesta las llamadas, construye los links de búsqueda, guarda "me gusta" en localStorage.

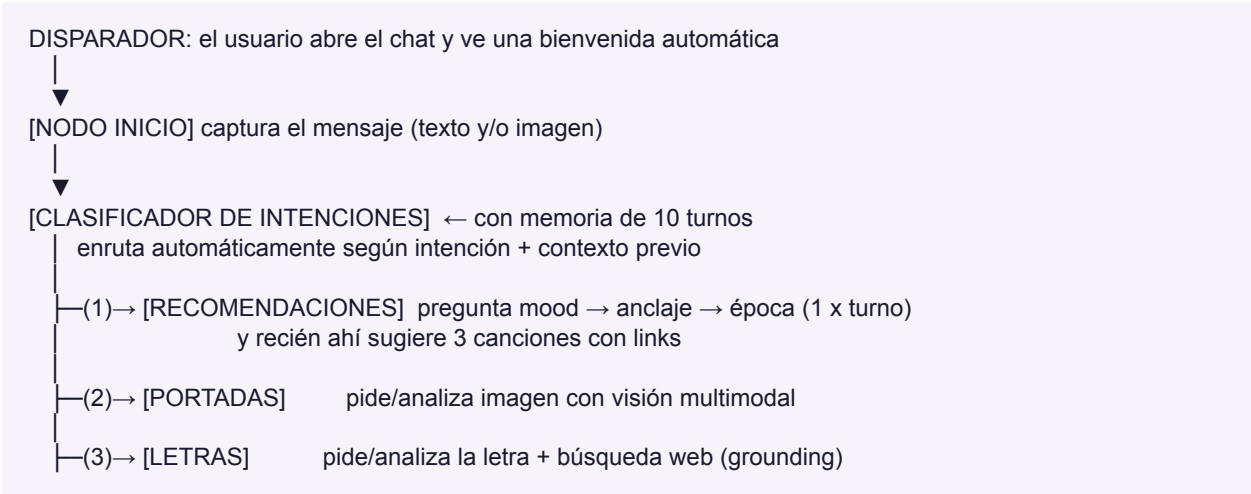
Dónde estaba la fricción

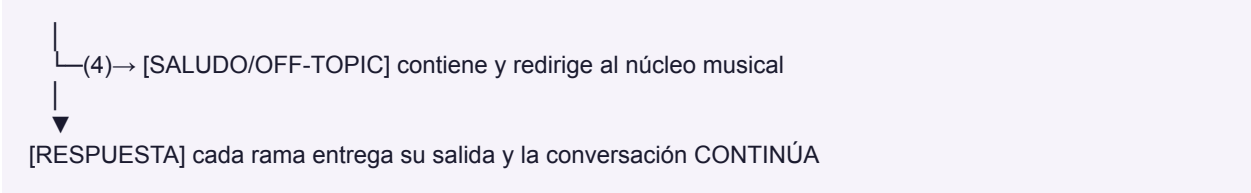
- **Interacción de un solo disparo:** el usuario debía anticipar y volcar todo su contexto en un formulario. No había forma de refinar en diálogo ("algo más tranquilo", "cambiá la época") sin reiniciar.
- **Sin memoria entre acciones:** cada función arrancaba en blanco. El sistema no sabía qué se había pedido dos minutos antes.
- **Carga cognitiva de navegación:** era el usuario quien decidía a qué pestaña ir. La responsabilidad de enrutar recaía sobre la persona, no sobre el sistema.
- **Rigidez estructural:** el formato de formulario impedía la exploración natural. La música se descubre conversando ("¿y algo parecido pero más viejo?"), no llenando campos.
- **Costo de mantenimiento técnico:** cada cambio de comportamiento implicaba tocar código TypeScript, redeployar y probar. La lógica vivía dispersa entre prompts y funciones.

c. El proceso, ahora (SintonIA)

SintonIA reconstruye el mismo proceso como un **Chatflow conversacional en Dify**. El cambio de fondo es que la experiencia deja de ser una serie de formularios aislados y pasa a ser **una sola conversación continua** en la que el sistema recuerda, pregunta y decide a dónde va cada mensaje.

Flujo rediseñado





Qué se eliminó, se aceleró o se delegó

Antes (CheTunes)	Ahora (SintonIA)	Efecto
El usuario elige la pestaña	El clasificador enruta solo	Delegado al sistema
Formulario de un disparo	Preguntas de seguimiento en diálogo	Rediseñado a conversación
Sin memoria entre acciones	Memoria de sesión en cada nodo	Agregado como valor central
Lógica en código TypeScript	Nodos visuales en Dify	Simplificado el mantenimiento
3 funciones separadas	3 ramas de un mismo flujo	Unificado el proceso

Herramientas que sostienen el proceso nuevo

Herramienta	Rol específico
Gemini 3.5 Flash	Modelo usado al inicio del proyecto, en la etapa de ideación, para detectar qué proceso de CheTunes convenía automatizar y cómo encarar el rediseño.
Claude Opus 4.6	Modelo usado en una etapa intermedia del armado del flujo. Con este nivel aparecían errores de configuración (IDs de nodo, clasificador sin memoria) que se detallan en Metodología.
Claude Opus 4.8	Modelo de IA usado como copiloto para generar y depurar la versión final del flujo de Dify (archivo DSL/YAML). Resolvió los errores de configuración anteriores.
Dify (Chatflow)	Plataforma visual que orquesta todo el flujo: nodos, memoria, enrutamiento y respuesta.
gpt-4o-mini (OpenAI)	Modelo base de 4 de los 5 nodos LLM (clasificador, recomendaciones, portadas y off-topic). Elegido por bajo costo, velocidad y visión multimodal.
Gemini 2.5 Flash + Grounding	Modelo del nodo de Análisis de Letras. El grounding le da búsqueda web de Google en tiempo real para identificar la canción real y evitar alucinaciones.
Question Classifier (Dify)	Nodo nativo que clasifica cada mensaje en una de 4 intenciones con memoria de contexto.
Visión multimodal	Habilitada en el nodo de portadas para leer y reconocer las imágenes de álbumes.

Decisión de arquitectura híbrida: no todos los nodos usan el mismo modelo. Cada nodo usa el modelo óptimo para su tarea: gpt-4o-mini donde funcionaba bien y era barato, y Gemini 2.5 Flash con grounding solo en Letras, que era el único nodo que necesitaba búsqueda web real para no inventar canciones.

d. Metodología

El proyecto usó distintos modelos de IA en cada etapa según lo que cada momento requería. Al inicio, en la fase de ideación, recurrí a **Gemini 3.5 Flash** para detectar cuál de los procesos de CheTunes convenía automatizar y cómo encarar el rediseño; esta decisión se tomó porque ya había usado este modelo en el trabajo anterior. Ya en el armado del flujo de Dify, utilicé **Claude** como copiloto para generar y depurar el archivo de configuración (formato YAML/DSL): primero con una versión de nivel medio (**Claude Opus 4.6**), con la que aparecían errores de configuración recurrentes —IDs de nodo mal interpretados, el clasificador sin memoria, prompts que fallaban al importar—, y luego con **Claude Opus 4.8**, que resolvió estos problemas y permitió llegar a la versión final estable. En todos los casos, cada decisión de arquitectura y de comportamiento del proceso fue mía: los modelos proponían y ejecutaban, pero yo dirigía. El desarrollo fue iterativo: cada versión del flujo se probó en la vista previa de Dify, se detectaron errores concretos y se corrigieron sobre el diagnóstico, no a ciegas.

Decisiones clave que se tomaron

- Definir que el valor central del rediseño era la **memoria conversacional**, y no una mejora estética o una función nueva.
- Elegir **gpt-4o-mini** como modelo base por sobre modelos más caros, priorizando costo y velocidad, ya que el proceso no requería más potencia.
- Decidir que el enrutamiento debía ser **responsabilidad del sistema** (clasificador) y no del usuario (pestañas).
- Agregar una **cuarta rama** (saludos/off-topic) que CheTunes no tenía, para contener mensajes fuera de foco sin romper la experiencia.
- Adoptar una **arquitectura híbrida de modelos**: usar Gemini 2.5 Flash con grounding solo en el nodo de Letras, y mantener gpt-4o-mini en el resto.
- Configurar **temperaturas distintas por nodo**: 0 en el clasificador (decisión determinista), y valores más bajos (0.3) en los nodos donde la precisión importaba más que la creatividad.

Qué salió mal y cómo lo corregí

Problema detectado	Diagnóstico	Corrección
La respuesta mostraba texto crudo tipo {{#node.text#}}	Dify no resolvía IDs de nodo con guiones; espera IDs numéricos	Reescribí todos los IDs a formato numérico nativo de Dify
El flujo de recomendaciones no hacía preguntas	Los prompts con caracteres acentuados fallaban al importar	Normalicé los prompts a texto sin tildes; el modelo igual responde con acentos
Responder "Radiohead" desviaba al flujo de portadas	El clasificador no tenía memoria: leía cada mensaje aislado	Activé memoria en el clasificador y agregué una regla de continuidad conversacional
Las preguntas de recomendación se repetían iguales	El prompt tenía preguntas fijas que el modelo copiaba literal	Reemplacé por un banco de 27 enfoques con instrucción de generar variantes únicas
Error de JSON al pegar letras largas	El max_tokens del clasificador era muy bajo y el JSON se truncaba	Subí el max_tokens del clasificador para que la respuesta se cierre completa

Problema detectado	Diagnóstico	Corrección
El análisis de letras inventaba canciones	gpt-4o-mini no puede verificar la canción real sin búsqueda web	Cambié el nodo de Letras a Gemini 2.5 Flash con grounding (búsqueda de Google)

Aprendizaje metodológico: heredé de CheTunes el principio de "divide y vencerás". Igual que allá separé identificación y análisis de letras en dos pasos, acá separé el enrutamiento (clasificador) de la ejecución (nodos), y di a cada nodo una sola responsabilidad clara. La misma lógica me llevó a la arquitectura híbrida: en vez de forzar un solo modelo para todo, le di a cada nodo el modelo que su tarea necesitaba.

e. Análisis crítico

Fortalezas detectadas

- **La conversación reemplaza al formulario:** el usuario ya no anticipa todo; construye su pedido en diálogo, que es como se descubre música de verdad.
- **Mantenimiento sin código:** cambiar el comportamiento del bot es editar un prompt en una interfaz visual, no reescribir y redeployar TypeScript.
- **Enrutamiento invisible:** el usuario escribe libremente y el sistema decide; desaparece la carga de elegir pestaña.
- **Arquitectura híbrida eficiente:** cada nodo usa el modelo justo para su tarea, optimizando costo donde se puede y precisión donde hace falta (búsqueda web solo en Letras).
- **Portabilidad:** todo el proceso vive en un único archivo DSL importable, versionable y compatible.

Limitaciones y desafíos

- **Memoria de sesión, no persistente:** SintonIA recuerda dentro de la conversación, pero no entre sesiones distintas. CheTunes sí guardaba "me gusta" en localStorage.
- **Dependencia del clasificador:** si el nodo de intención se equivoca, toda la rama posterior falla. Es un punto único de decisión.
- **Costo y latencia del grounding:** la búsqueda web en el nodo de Letras mejora la precisión pero hace esa respuesta más lenta y algo más cara. Es el mismo trade-off que en CheTunes con googleSearch.
- **Sin reproducción real:** igual que CheTunes, se entregan links de búsqueda, no streaming embebido (limitación de seguridad de las APIs).
- **Dependencia del proveedor de IA:** si la API del modelo cae, el proceso queda inoperativo.

Oportunidades futuras

- Extender el grounding a los nodos de Recomendaciones y Portadas para verificar también canciones sugeridas y datos de álbumes.
- Incorporar variables de conversación persistentes en Dify para recuperar el "aprendizaje" de gustos que tenía CheTunes.
- Sumar un nodo de feedback ("¿te sirvió esta recomendación?") que alimente las siguientes.

Evaluación con las cuatro dimensiones AIBPS

Agilidad — ¿Es más rápido?

Sí en la exploración iterativa: ajustar un pedido antes tomaba reiniciar el formulario; ahora es un mensaje más en el chat. La memoria elimina la recarga de contexto. Como contrapartida, el flujo conversacional puede requerir más turnos que un formulario único cuando el usuario ya tiene todo claro, y el nodo de Letras con grounding tarda un poco más porque busca en la web antes de responder. La agilidad se gana en la exploración, no en el atajo directo.

Fluidez — ¿Es fácil de operar?

Mejora clara. Desaparece la fricción de elegir pestaña y anticipar campos. La interacción imita una charla natural con un amigo que sabe de música. El riesgo de fricción nueva —que el clasificador malinterprete— se mitigó con memoria y reglas de continuidad conversacional.

Protección — ¿Los datos están resguardados y son trazables?

El proceso no maneja datos personales sensibles: la conversación es efímera y no se almacenan perfiles permanentes. Cada ejecución queda registrada en los logs de Dify, lo que aporta trazabilidad. La contracara es que no hay persistencia intencional de datos del usuario, lo que protege la privacidad pero limita la personalización a largo plazo.

Control — ¿Quién tiene la última palabra? ¿Se puede auditar?

El control de diseño es humano: yo definí las reglas de enrutamiento, los límites de cada nodo y los prompts, incluida la decisión de qué modelo usa cada rama. La IA actúa dentro de esos límites y nunca ejecuta acciones irreversibles: solo sugiere música, links y análisis. El usuario siempre valida la recomendación final decidiendo si la escucha. El flujo es auditable nodo por nodo en la interfaz visual de Dify.

f. Conclusiones

El paso de CheTunes a SintonIA demuestra que **rediseñar la capa de interacción de un proceso puede aportar más valor que agregar funciones nuevas**. Las tres funcionalidades son las mismas; lo que cambió es cómo la persona se relaciona con ellas. Pasar del disparo único a la conversación con memoria convirtió una herramienta útil en una experiencia fluida.

Aprendizajes principales

1. **El proceso importa más que la función:** no hacía falta una capacidad nueva, sino rediseñar cómo se accedía a las existentes. El diagnóstico del "antes" fue lo que reveló que la fricción estaba en la interacción, no en las prestaciones.
2. **El bajo código cambia quién puede iterar:** migrar de TypeScript a Dify significó que ajustar el comportamiento del proceso ya no requiere ser programador, sino entender el proceso.
3. **No hay un modelo único para todo:** la arquitectura híbrida enseñó que conviene elegir el modelo según la tarea de cada nodo. gpt-4o-mini alcanzaba para la mayoría, pero solo Gemini con grounding resolvió la alucinación en Letras.

4. **La memoria es infraestructura, no adorno:** darle contexto al sistema fue lo que transformó tres formularios aislados en una sola conversación coherente.

¿Qué haría distinto y hasta dónde escalaría?

Sumaría persistencia entre sesiones para recuperar la personalización que CheTunes tenía con localStorage, y extendería la búsqueda web (grounding) a Recomendaciones y Portadas para eliminar la alucinación en todo el flujo, no solo en Letras. En cuanto a escala, este mismo patrón (bienvenida → clasificador → ramas especializadas con memoria y modelo óptimo por tarea) es replicable a cualquier asistente de dominio: soporte, trámites, atención comercial. Lo que aprendí rediseñando un bot de música es, en el fondo, un patrón de proceso reutilizable.

De una web que respondía una vez, a una conversación que acompaña. Ese fue el verdadero rediseño.